

Mi Ruta de Hábitos

Jetpack Compose + Room Database

¿qué vamos a hacer?

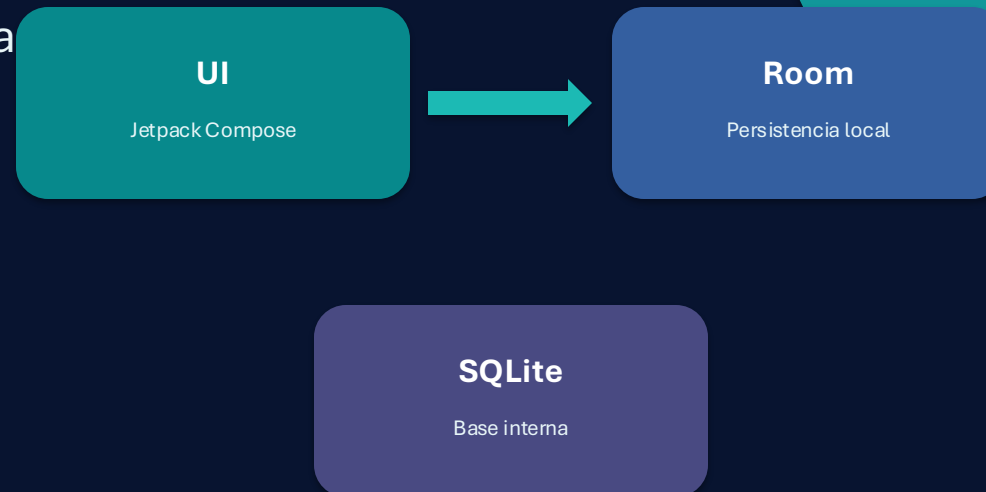
Guardar hábitos en una base de datos local para que permanezcan aunque la app se cierre.



Ruta de aprendizaje

De una app en memoria a una app con persistencia real

- Entender el problema: los datos se pierden al cerrar la app.
- Agregar Room al proyecto con dependencias y KSP.
- Crear Entity, DAO y Database.
- Conectar Room con Jetpack Compose.
- Insertar, consultar, actualizar y eliminar hábitos.
- Proponer retos para extender el proyecto.



El problema del proyecto original

La lista funcionaba, pero solo vivía en memoria

Antes: estado en memoria

```
var listaHabitos by remember {  
    mutableStateOf(  
        listOf(  
            Habito("Tomar agua"),  
            Habito("Leer 10 minutos"),  
            Habito("Estudiar Kotlin")  
        )  
    )  
}
```

¿Qué significa?

remember conserva datos mientras la pantalla está activa, pero no guarda información permanente en el dispositivo.

Necesidad

Al cerrar y volver a abrir la app, los hábitos deben seguir disponibles.

¿Qué es Room Database?

Una capa moderna para trabajar con SQLite desde Kotlin

- Room permite crear bases de datos locales de forma segura.
- Reduce errores comunes al escribir SQL manualmente
- Se integra con Kotlin, Flow, corrutinas y Jetpack Compose
- Es ideal para apps que necesitan guardar información

Entity

Define la tabla

DAO

Define operaciones

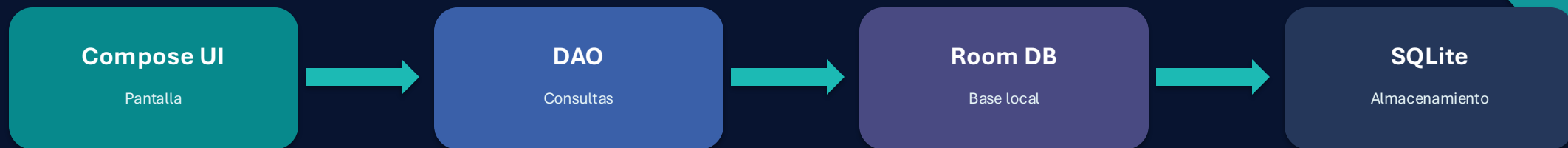
Database

Conecta todo

Ejemplo mental: Room es el puente entre tu interfaz Compose y la base SQLite interna del teléfono.

Arquitectura que usaremos

Flujo de datos de la app



- La UI no escribe directamente en SQLite.
- Compose solicita acciones al DAO.
- Room traduce esas acciones a operaciones sobre la base local.
- Flow devuelve cambios hacia la UI en tiempo real.

Paso 1 · Agregar dependencias

Room necesita runtime, KTX, compilador y KSP

```
build.gradle.kts (:app)

plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    id("com.google.devtools.ksp")
}

dependencies {
    val roomVersion = "2.8.4"

    implementation("androidx.room:room-runtime:$roomVersion")
    implementation("androidx.room:room-ktx:$roomVersion")
    ksp("androidx.room:room-compiler:$roomVersion")
}
```

Explicación

- room-runtime: habilita Room.
- room-ktx: soporte para Kotlin, corrutinas y Flow.
- room-compiler: genera código interno.
- KSP: procesa anotaciones de Kotlin de forma eficiente.

Paso 2 · Crear la entidad

La entidad representa una tabla de la base de datos

HabitoEntity.kt

```
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "habitos")
data class HabitoEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val nombre: String,
    val completado: Boolean = false
)
```

Lectura del código

- @Entity crea la tabla habitos.
- @PrimaryKey identifica cada fila.
- autoGenerate = true crea ids automáticamente.
- nombre guarda el texto del hábito.
- completado guarda true/false.

Ejemplo

Si el usuario escribe “Caminar 20 minutos”, Room guardará una fila con id automático, nombre = Caminar 20 minutos y completado = false.

Así se vería la tabla

Room convierte la clase HabitoEntity en una tabla local

id	nombre	completado
1	Tomar agua	false
2	Leer 10 minutos	true
3	Estudiar Kotlin	false

- Cada objeto HabitoEntity equivale a un registro.
- El campo id ayuda a distinguir hábitos aunque tengan nombres parecidos.
- El campo completado permite marcar o desmarcar el hábito.

Paso 3 · Crear el DAO

El DAO define cómo consultar y modificar los hábitos

HabitoDao.kt

```
import androidx.room.*
import kotlinx.coroutines.flow.Flow

@Dao
interface HabitoDao {

    @Query("SELECT * FROM habitos")
    fun obtenerHabitos(): Flow<List<HabitoEntity>>

    @Insert
    suspend fun insertarHabito(habito: HabitoEntity)

    @Update
    suspend fun actualizarHabito(habito: HabitoEntity)

    @Delete
    suspend fun eliminarHabito(habito: HabitoEntity)
}
```

Explicación por anotaciones

- @Dao: marca la interfaz de acceso a datos.
- @Query: ejecuta una consulta SQL.
- @Insert: inserta un nuevo registro.
- @Update: actualiza un registro existente.
- @Delete: elimina un registro.
- suspend: se ejecuta con corrutinas.

¿Por qué usamos Flow?

Para que la lista se actualice cuando cambian los datos

Consulta reactiva

```
@Query("SELECT * FROM habitos")  
fun obtenerHabitos():  
    Flow<List<HabitoEntity>>
```

- Flow emite una nueva lista cuando cambia la tabla.
- Compose recibe esos cambios con `collectAsState()`.
- Ahora no necesitas refrescar manualmente la pantalla.



Paso 4 · Crear la base de datos

HabitoD

HabitoDatabase.kt

```
@Database(
    entities = [HabitoEntity::class],
    version = 1
)
abstract class HabitoDatabase : RoomDatabase() {

    abstract fun habitoDao(): HabitoDao

    companion object {
        @Volatile
        private var INSTANCE: HabitoDatabase? = null

        fun obtenerBaseDatos(context: Context): HabitoDatabase {
            return INSTANCE ?: synchronized(this) {
                val instancia = Room.databaseBuilder(
                    context.applicationContext,
                    HabitoDatabase::class.java,
                    "habitos_db"
                ).build()

                INSTANCE = instancia
                instancia
            }
        }
    }
}
```

Puntos importantes

- @Database registra las entidades.
- version = 1 indica la primera versión.
- abstract fun habitoDao() expone el DAO.
- INSTANCE evita crear varias bases.
- synchronized protege la creación en simultáneo.
- databaseBuilder crea hábitos_db.

¿Por qué usamos Singleton?

Para tener una sola instancia de la base de datos

Patrón Singleton

```
@Volatile
private var INSTANCE: HabitoDatabase? = null

return INSTANCE ?: synchronized(this) {
    val instancia = Room.databaseBuilder(
        context.applicationContext,
        HabitoDatabase::class.java,
        "habitos_db"
    ).build()

    INSTANCE = instancia
    instancia
}
```

Analogía

La base de datos es como la libreta oficial del grupo: todos deben escribir en la misma libreta, no en copias diferentes.

- Evita múltiples conexiones innecesarias.
- Mejora el rendimiento.
- Mantiene consistencia en los datos.

Paso 5 · Conectar la base en MainActivity

La actividad crea la base y pasa el DAO a la UI

```
MainActivity.kt

class MainActivity : ComponentActivity() {

    private lateinit var database: HabitoDatabase

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        database =
            HabitoDatabase.obtenerBaseDatos(this)

        setContent {
            MiRutaHabitosApp(database.habitoDao())
        }
    }
}
```

Explicación

- database se inicializa al iniciar la app.
- obtenerBaseDatos(this) crea o recupera la base.
- setContent carga la interfaz Compose.
- MiRutaHabitosApp recibe el habitoDao.

Esto separa la UI del acceso directo a la base.

Paso 6 · Leer datos desde Compose

Compose observa la base con `collectAsState()`

```

MiRutaHabitosApp()

@Composable
fun MiRutaHabitosApp(habitoDao: HabitoDao) {
    var nuevoHabito by remember { mutableStateOf("") }

    val listaHabitos by habitDao
        .obtenerHabitos()
        .collectAsState(initial = emptyList())

    val scope = rememberCoroutineScope()

    val completados = listaHabitos.count { it.completado }
}
```

Qué ocurre aquí

- `nuevoHabito` guarda el texto del `TextField`.
- `obtenerHabitos()` regresa un `Flow`.
- `collectAsState()` convierte el `Flow` en estado de Compose.
- `scope` permite ejecutar operaciones suspend.
- `completados` calcula el resumen.

Idea clave

Cuando Room detecta cambios, `Flow` emite una nueva lista y Compose recompone la pantalla automáticamente.

Paso 7 • Estructura visual de la pantalla

La UI mantiene la lógica del proyecto original

```
Estructura Compose

MaterialTheme {
    Surface(modifier = Modifier.fillMaxSize()) {
        Column(
            modifier = Modifier
                .padding(20.dp)
                .fillMaxSize()
        ) {
            Text("Mi Ruta de Hábitos")
            Text("Hábitos completados: $completados de
                ${listaHabitos.size}")
            OutlinedTextField(...)
            Button(...)
            LazyColumn { ... }
        }
    }
}
```

Componentes usados

- MaterialTheme: estilo general.
- Surface: contenedor principal.
- Column: organiza verticalmente.
- Text: muestra títulos y resumen.
- OutlinedTextField: captura entrada.
- Button: inserta datos.
- LazyColumn: muestra lista dinámica.

Paso 8 · Insertar un hábito

El botón guarda un nuevo registro en Room

● ● ● Botón agregar

```
Button(  
    onClick = {  
        if (nuevoHabitito.isNotEmpty()) {  
            scope.launch {  
                habititoDao.insertarHabitito(  
                    HabititoEntity(nombre = nuevoHabitito)  
                )  
                nuevoHabitito = ""  
            }  
        }  
    },  
    modifier = Modifier.fillMaxWidth()  
) {  
    Text("Agregar hábito")  
}
```

Secuencia de ejecución

1. El usuario escribe un hábito.
2. Se valida que no esté vacío.
3. scope.launch inicia una corrutina.
4. insertarHabitito() guarda en Room.
5. nuevoHabitito = "" limpia el campo.
6. Flow actualiza la lista.

Paso 9 • Mostrar, actualizar y eliminar

Code LazyColumn conectada a Room

```
LazyColumn {
    items(listaHabitos) { habito ->
        HabitoCard(
            habito = habito,
            onCambioEstado = {
                scope.launch {
                    habitoDao.actualizarHabitto(
                        habito.copy(
                            completado = !habitito.completado
                        )
                    )
                }
            },
            onEliminar = {
                scope.launch {
                    habititoDao.eliminarHabitto(habitito)
                }
            }
        )
    }
}
```

Lectura didáctica

- items(listaHabitos) dibuja cada registro.
- onCambioEstado actualiza completado.
- copy() crea una versión modificada.
- onEliminar borra el registro.
- Cada operación dispara una nueva emisión de Flow.

Detalle importante · `copy()`

Modificar datos sin perder el objeto completo

```
habito.copy(  
    completado =  
    !habito.completado  
)
```

- Si estaba pendiente, pasa a completado.
- Si estaba completado, pasa a pendiente.
- Room identifica el registro gracias al id.

Explicación

`copy()` crea una copia del hábito actual, conservando id y nombre, pero cambiando el campo completado.

Paso 10 · Crear la tarjeta de hábito

```

@Composable
fun HabitoCard(
    habito: HabitoEntity,
    onCambioEstado: () -> Unit,
    onEliminar: () -> Unit
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 6.dp),
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
    ) {
        Row(
            modifier = Modifier
                .padding(16.dp)
                .fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Column(modifier = Modifier.weight(1f)) {
                Text(text = habito.nombre)
                Text(text = if (habito.completado) "Completado" else "Pendiente")
            }

            Checkbox(
                checked = habito.completado,
                onCheckedChange = { onCambioEstado() }
            )

            TextButton(onClick = onEliminar) {
                Text("Eliminar")
            }
        }
    }
}

```

Qué muestra la tarjeta

- Nombre del hábito.
- Estado: Completado o Pendiente.
- Checkbox para cambiar estado.
- Botón Eliminar.

Separar este componente hace que el código sea más limpio y fácil de mantener.

Flujo completo de la app

Lo que pasa cuando el estudiante interactúa con la pantalla

1. Escribe

TextField

2. Presiona

Button

3. Inserta

DAO

4. Guarda

Room

5. Emite

Flow

6. Actualiza

Compose

Ok, ok...

La interfaz no se redibuja manualmente. Cambia el dato, Room emite una nueva lista y Compose actualiza la pantalla.

Organización recomendada de archivos

Para que el proyecto sea claro desde el inicio

Estructura sugerida

```
com.example.mirutahabitos
├── MainActivity.kt
├── data
│   ├── HabitoEntity.kt
│   ├── HabitoDao.kt
│   └── HabitoDatabase.kt
└── ui
    ├── MiRutaHabitosApp.kt
    └── HabitoCard.kt
```

Nota mental:

Al principio se puede mantener todo en el mismo paquete. Después, conviene separar data y ui para acercarse a una arquitectura más profesional.

Antes vs Ahora

Evolución del proyecto

Antes: solo memoria

- Lista creada con remember.
- Los datos viven temporalmente.
- No hay base local.
- La app se reinicia limpia.
- Ideal para aprender estado inicial.

Ahora: Room Database

- Lista guardada en SQLite.
- Los datos permanecen.
- DAO controla las operaciones.
- Flow actualiza la UI.
- Se acerca a una app real.

Errores comunes y solución

Para acompañar la práctica en laboratorio

- Error con KSP: revisar que el plugin id("com.google.devtools.ksp") esté agregado.
- No reconoce Room: sincronizar Gradle después de agregar dependencias.
- La app no compila: revisar imports de Room, Flow y corrutinas.
- La lista no se actualiza: confirmar que obtenerHabitos() regrese `Flow<List<HabitoEntity>>`.
- La operación marca error por suspend: ejecutar dentro de `scope.launch`.

Reto 1 · Eliminar hábitos completados

Extensión sencilla para reforzar DAO + botón

• • • Agregar en HabitoDao.kt

```
@Query("DELETE FROM habitos WHERE  
completado = 1")  
suspend fun eliminarCompletados()
```

• • • Agregar en Compose

```
Button(  
    onClick = {  
        scope.launch {  
            habitoDao.eliminarCompletados()  
        }  
    }  
) {  
    Text("Eliminar completados")  
}
```

Cosas para revisar:

- ¿Dónde debe ir la consulta SQL?
- ¿Por qué es suspend?
- ¿Qué pasa con la UI después de eliminar?
- ¿Por qué no necesitamos refrescar manualmente la lista?

¿Qué sigue?

Para ir más adelante

A

- Validar hábitos repetidos.
- Agregar mensaje de error.
- Limpiar espacios con trim().

B

- Agregar categoría.
- Filtrar por pendientes.
- Ordenar por completados.

C

- Usar ViewModel.
- Separar repository.
- Crear navegación.
- Aplicar MVVM.

Conceptos dominados

Compose · Estado · Room · Entity · DAO · Database · Flow · Corrutinas

